



Alma Mater Studiorum-Università di Bologna Scuola di Ingegneria

Esercitazione Calcolo dei seggi con metodo proporzionale

Corso di Laurea in Ingegneria Informatica
Anno accademico 2023/2024

Prof. ENRICO DENTI

Dipartimento di Informatica – Scienza e Ingegneria (DISI)

IL PROBLEMA: DAI VOTI AI SEGGI

- In qualunque elezione, gli elettori votano per una lista: quei *voti* devono essere *trasformati in seggi* nel modo stabilito dal *sistema elettorale*.
- In un sistema *proporzionale*, ciò avviene assegnando i seggi *in proporzione* ai voti ottenuti
 - MA cosa significa di preciso "*in proporzione*"?
 - una proporzione matematica esatta dà spesso luogo a un *numero non intero di seggi*, quindi occorre in qualche modo "arrotondarli"
- I diversi modi di farlo danno luogo a sistemi elettorali diversi
 - sistema "ai quozienti interi e ai più alti resti" (Hare)
 - sistema D'Hondt
 - sistema Saint-Lague
 - ... e altri..

METODO PROPORZIONALE AI QUOZIENTI INTERI E PIÙ ALTI RESTI

- In questo sistema si agisce in due tempi
 1. **in prima battuta** si attribuiscono i seggi in proporzione ai voti ottenuti arrotondando i risultati all'intero precedente
→ si assegnano perciò, in generale, *meno seggi* del dovuto
 2. **in seconda battuta** si assegnano i seggi rimasti *attribuendoli alle liste coi più alti resti* (ossia col maggior numero di voti "inutilizzati")

Per attribuire i seggi in proporzione (Step 1):

- Si prendono i voti totali V ottenuti da tutte le liste ESEMPIO: 423.000
- Si considerano i seggi totali S da assegnare ESEMPIO: 47
- Si calcola il **quoziente elettorale Q** dividendo V/S che *rappresenta il costo (in voti) di un seggio* ESEMPIO: $423.000/47 = 9.000$
- Si calcolano i **seggi spettanti a ciascuna lista** dividendo i voti di ciascuno per il quoziente Q , ovvero $\text{Seggi } [i] = \text{Voti } [i] / \text{Quoziente}$ ESEMPIO:
seggi $[A] = \text{voti } [A] / 9.000$
seggi $[B] = \text{voti } [B] / 9.000$
...etc.

METODO PROPORZIONALE AI QUOZIENTI INTERI E PIÙ ALTI RESTI

- In questo sistema si agisce in due tempi
 1. **in prima battuta** si attribuiscono i seggi in proporzione ai voti ottenuti arrotondando i risultati all'intero precedente
→ si assegnano perciò, in generale, *meno seggi* del dovuto
 2. **in seconda battuta** si assegnano i seggi rimasti *attribuendoli alle liste coi più alti resti* (ossia col maggior numero di voti "inutilizzati")

ESEMPIO: caso limite con tutti i seggi assegnati in proporzione esatta

- Voti totali: 423.000
 - Seggi da assegnare: 47
 - Quoziente: $423.000/47 = 9.000$
 - Seggi [i] = Voti [i] / Quoziente
- Sono assegnati tutti i 47 seggi ☺
→ *Passo 2 non necessario*

	Voti ottenuti		Seggi assegnati
Lista A	171.000		19
Lista B	135.000		15
Lista C	81.000	: 9.000	9
Lista D	36.000		4
	423.000		47

METODO PROPORZIONALE AI QUOZIENTI INTERI E PIÙ ALTI RESTI

- In questo sistema si agisce in due tempi
 - in prima battuta si attribuiscono i seggi in proporzione ai voti ottenuti *arrotondando i risultati all'intero precedente*
→ si assegnano perciò, in generale, *meno seggi* del dovuto
 - in seconda battuta** si assegnano i seggi rimasti *attribuendoli alle liste coi più alti resti* (ossia col maggior numero di voti "inutilizzati")

ESEMPIO: caso realistico con **non tutti i seggi** assegnati al Passo 1

- Voti totali: 423.000
 - Seggi da assegnare: 47
 - Quoziente: $423.000 / 47 = 9.000$
 - Seggi [i] = Voti [i] / Quoziente
- Sono assegnati **solo 46 seggi** ☺
→ **Passo 2 necessario**

	Voti ottenuti	Seggi assegnati
Lista A	171.000	19
Lista B	132.000	14
Lista C	84.000	9
Lista D	36.000	4
	423.000	46

14,67

9,33

: 9.000

METODO PROPORZIONALE AI QUOZIENTI INTERI E PIÙ ALTI RESTI

- Per calcolare i resti (ossia i voti residui non ancora utilizzati), *per ogni lista*:
 - si calcolano dapprima i voti realmente utilizzati, ri-moltiplicando il quoziente elettorale Q per i seggi finora assegnati a tale lista
 - si sottraggono tali voti da quelli ottenuti dalla lista
- Poi, **si assegnano i seggi residui alle liste coi più alti resti**

ESEMPIO: c'è un solo seggio residuo in palio, lo si dà alla lista B

	Voti ottenuti	Seggi al quoziente	Seggi assegnati	Resti	Seggi ai resti	Tot. Seggi
Lista A	171.000	19,00	132.000 -9.000 x 14	0	0	19
Lista B	132.000	14,67	14	6.000	1	15
Lista C	84.000	9,33	9	3.000	0	9
Lista D	36.000	4,00	84.000 -9.000 x 9	0	0	4
	423.000					47



IL SISTEMA SOFTWARE

Prima versione

- *Questione di fondo: come accoppiare partiti e voti?*
 - i **nomi delle liste** siano contenuti in un array di stringhe (**partiti**)
 - i **voti ottenuti** dalle varie liste siano posti in un array di long (**voti**)
 - IPOTESI: **voti[i]** contiene i voti della lista di nome **partiti[i]**
 - ergo, anche i **voti residui (resti)** saranno in un array di long (**resti**)
- *Punti di attenzione:*
 - il **calcolo del quoziente** effettua una divisione fra long
MA il risultato dev'essere reale → serve una conversione ☹
double quoziente = (double) votiTotali / seggiDaAssegnare
 - dualmente, il **calcolo dei seggi** effettua una divisione fra reali
MA il risultato dev'essere intero → arrotondamento per difetto con **Math.floor** e successiva conversione in **long**
seggi[i] = (long) Math.floor(voti[i] / quoziente)

IL SISTEMA SOFTWARE

Prima versione

- Questione di fondo: *come accoppiare partiti e voti?*

- i **nomi delle liste** siano contenuti in un array di stringhe (**partiti**)
- i **voti ottenuti** dalle varie liste siano posti in un array di long (**voti**)
- IPOTESI: **voti[i]** contiene i voti della lista di nome **partiti[i]**
- e per i voti residui (**resti**) saranno in un array di long (**resti**)

Semplice, immediata, ma...
è davvero una buona idea?

partiti	Lista A	Lista B	Lista C	Lista D	
			i		
voti	171.000	132.000	84.000	36.000	

In realtà, NO: spezzare le informazioni in due array **da tenere sincronizzati con un indice comune** avrà il solo effetto di *farcì impazzire a livello di gestione*

La solita vecchia storia: **rappresentazione naif = più codice**
Approccio migliore: **rappresentazione più sofisticata = meno codice**

IL SISTEMA SOFTWARE

Prima versione

- Architettura del sistema
 - una classe **CalcolatoreSeggi** incapsula la logica di calcolo
 - metodo **calcolaSeggi**
 - il cliente (main)
 - *prima* crea il **CalcolatoreSeggi**
 - *poi* gli chiede di calcolare i seggi

Specifica di dettaglio

- il metodo **calcolaSeggi** riceve
 - il numero di seggi da assegnare (**long**)
 - l'array coi nomi delle liste (**String[]**)
 - l'array coi voti da esse ottenuti (**long[]**)
- e produce come risultato
 - l'array coi seggi assegnati (**long[]**)

model::CalcolatoreSeggi	
-	nome: String
-	sb: StringBuilder
	arrayToString(long[]): String
+	calcolaSeggi(long, String[], long[]): long[]
+	CalcolatoreSeggi(String, StringBuilder)
+	getLogger(): StringBuilder
+	getNome(): String
-	getVotiTotali(long[]): long
-	trovaPosMax(long[]): int

Passiamo al costruttore uno **StringBuilder** in cui accumulare il log dell'attività
No stampe dirette a video!!

IL SISTEMA SOFTWARE

Prima versione

Java

```
package elections.model;

public class CalcolatoreSeggi {
    private String nome;
    private StringBuilder sb;
    public CalcolatoreSeggi(String nome, StringBuilder sb){
        this.nome=nome; this.sb=sb;
    }
    public String getNome() {
        return nome;
    }
    public StringBuilder getLogger() {
        return sb;
    }
    public long[] calcolaSeggi(
        long seggiDaAssegnare, String[] partiti, long[] voti) {
        ...
    }
}
```

Nome di questo calcolatore
(algoritmo)

StringBuilder in cui
accumulare il log dell'attività

recuperabile dall'esterno per comodità

Signature complicata e poco
parlante: scomodità in vista..

ALGORITMO DI MASSIMA

```
public long[] calcolaSeggi(long seggiDaAssegnare, String[] partiti, long[] voti)
{
    long votiTotali = getVotiTotali(voti);
    double quozienteElettorale = ...;
    int seggiAssegnati = 0;
    long[] seggi = ...;
    long[] resti = ...;
    for (int i=0; i<partiti.length; i++) {
        seggi[i] = ...;
        seggiAssegnati += seggi[i];
        resti[i] = ...;
    }
    ...
    // SEGUE
}
```

metodo privato ausiliario

da calcolare (occhio!)

da creare

1° passo

da calcolare (occhio!)

da calcolare (occhio!)

Java

ALGORITMO DI MASSIMA

```
public long[] calcolaSeggi(long seggiDaAssegnare, String[] partiti, long[] voti)
{
    ...
    // CONTINUA
    while (seggiDaAssegnare > seggiAssegnati) {
        int posMax = trovaPosMax(resti);
        seggi[posMax]++;
        seggiAssegnati++;
        resti[posMax] = 0;
    }
    return seggi;
}
```

Java

2° passo

metodo privato ausiliario

Una volta assegnato un seggio ausiliario, quella lista non va più considerata per nuove assegnazioni (perché si assegna al più un seggio ausiliario) → azzeriamo i voti residui per disattivarla

NB: in questo algoritmo, il numero di seggi residui da assegnare ai resti è sempre inferiore al numero di partiti, altrimenti sarebbe scattata un'assegnazione in più al passo precedente.

Di conseguenza, si può dimostrare che a ogni lista può essere assegnato sempre al più UN seggio ausiliario.

COLLAUDO ARTIGIANALE (SENZA JUnit)

- Architettura del sistema

- una classe **Setup** incapsula il main

- un metodo per ogni test

- il main

- crea lo **StringBuilder**
 - crea il **CalcolatoreSeggi**

- ogni metodo di test

- crea i tre array **partiti**, **voti**, **seggi**
 - invoca il metodo **calcolaSeggi** sul **CalcolatoreSeggi** ricevuto
 - **asserisce i risultati attesi** → **assert**
 - **VOLENDO**: stampa a video i risultati
 - **VOLENDO**: stampa a video il log del **CalcolatoreSeggi**

VOI potreste ristrutturarlo
con JUnit! 😊

main::Setup

```
+ main(String[]): void  
- test1(CalcolatoreSeggi): void  
- test2(CalcolatoreSeggi): void  
- test3(CalcolatoreSeggi): void
```



COLLAUDO ARTIGIANALE (SENZA JUnit)

Java

```
public class Setup {  
    public static void main(String[] args) {  
        StringBuilder sb = new StringBuilder();  
        CalcolatoreSeggi calculator = new CalcolatoreSeggi("proporzionale", sb);  
        test1(calculator);  
        test2(calculator);  
        test3(calculator);  
    }  
  
    private static void test1(CalcolatoreSeggi calculator) {  
        ... // calcoli  
        assert seggi[0] == ...;  
        assert seggi[1] == ...;  
        assert seggi[2] == ...;  
        ...  
        assert seggi[0]+seggi[1]+seggi[2]+seggi[3]+seggi[4] == SEGGI_DA_ASSEGNARE;  
        System.out.println(...); // VOLENDO  
    }  
}
```

Un domani potrebbe esserci anche
un altro algoritmo...
.. e magari un altro calcolatore...

COLLAUDO: TEST #1

```
private static void test1(CalcolatoreSeggi calculator) {  
    String[] partiti = { "Formiche rosse", "Topolini grigi", "Farfalle blu",  
                        "Bruchi verdi", "Canarini gialli"};  
    long[] voti = {10, 100, 200, 300, 400};  
    final long SEGGI_DA_ASSEGNARE = 100;  
    long[] seggi = calculator.calcolaSeggi(SEGGI_DA_ASSEGNARE, partiti, voti);  
    assert...;  
    System.out.println("PARTITO\t" + "\t" + "VOTI" + "\t" + "SEGGI");  
    for (int i=0; i<partiti.length; i++)  
        System.out.println(partiti[i] + "\t " + voti[i] + "\t " + seggi[i]);  
    System.out.println("---Log calcolo---" +  
                        System.lineSeparator() + calculator.getLogger());  
}
```

Java

Bleah

PARTITO	VOTI	SEGGI
Formiche rosse	10	1
Topolini grigi	100	10
Farfalle blu	200	20
Bruchi verdi	300	30
Canarini gialli	400	39

```
---Log calcolo---  
Voti presi: [10,100,200,300,400]  
Seggi da assegnare: 100  
Quoziente elettorale : 10.1  
Seggi assegnati al primo giro: [0,9,19,29,39]  
Resti dopo il primo giro: [10,9,8,7,6]  
Seggi ancora da assegnare: 4  
Seggi assegnati alla fine: [1,10,20,30,39]
```

COLLAUDO: TEST #2

```
private static void test1(CalcolatoreSeggi calculator) {
    String[] partiti = {"Lista A\t", "Lista B\t", "Lista C\t", "Lista D\t"};
    long[] voti = {171000,132000,84000,36000};
    final long SEGGI_DA_ASSEGNARE = 8;
    long[] seggi = calculator.calcolaSeggi(SEGGI_DA_ASSEGNARE, partiti, voti);

    assert...;

    System.out.println("PARTITO\t" + "\t" + "VOTI" + "\t" + "SEGGI");
    for (int i=0; i<partiti.length; i++)
        System.out.println(partiti[i] + "\t " + voti[i] + "\t " + seggi[i]);
    System.out.println("---Log calcolo---" +
        System.lineSeparator() + calculator.getLogger());
}
```

Java

Bleah

PARTITO	VOTI	SEGGI
Lista A	171000	3
Lista B	132000	2
Lista C	84000	2
Lista D	36000	1

```
---Log calcolo---
Voti presi: [171000,132000,84000,36000]
Seggi da assegnare: 8
Quoziente elettorale : 52875.0
Seggi assegnati al primo giro: [3,2,1,0]
Resti dopo il primo giro: [12375,26250,31125,36000]
Seggi ancora da assegnare: 2
Seggi assegnati alla fine: [3,2,2,1]
```

COLLAUDO: TEST #3

```
private static void test1(CalcolatoreSeggi calculator) {
    String[] partiti = {"Gialli", "Bianchi", "Rossi", "Verdi", "Blu", "Rosa"};
    long[] voti = {47000, 16000, 15800, 12000, 6100, 3100};
    final long SEGGI_DA_ASSEGNARE = 10;
    long[] seggi = calculator.calcolaSeggi(SEGGI_DA_ASSEGNARE, partiti, voti);

    assert...;

    System.out.println("PARTITO\t" + "\t" + "VOTI" + "\t" + "SEGGI");
    for (int i=0; i<partiti.length; i++)
        System.out.println(partiti[i] + "\t " + voti[i] + "\t " + seggi[i]);
    System.out.println("---Log calcolo---" +
        System.lineSeparator() + calculator.getLogger());
}
```

Java

Bleah

PARTITO	VOTI	SEGGI
Gialli	47000	5
Bianchi	16000	2
Rossi	15800	1
Verdi	12000	1
Blu	6100	1
Rosa	3100	0

```
---Log calcolo---
Voti presi: [47000,16000,15800,12000,6100,3100]
Seggi da assegnare: 10
Quoziente elettorale : 10000.0
Seggi assegnati al primo giro: [4,1,1,1,0,0]
Resti dopo il pr. giro: [7000,6000,5800,2000,6100,3100]
Seggi ancora da assegnare: 3
Seggi assegnati alla fine: [5,2,1,1,1,0]
```

IL SISTEMA SOFTWARE

Verso il refactoring

- BILANCIO: ce l'abbiamo fatta, ma.. a che prezzo!
- Il (non) *accoppiare direttamente partiti e voti ha reso il codice complicato, poco leggibile ed **error-prone***
 - siamo ammattiti a saltare continuamente da un array all'altro, *usando l'indice per tenere insieme partito, voti (e resti)*
- Come già constatato con l'esempio banconote/portafoglio, *quando c'è **troppo codice troppo complicato** è quasi sempre segno che la rappresentazione dei dati è **troppo naif***
- Ma se partito e voti costituiscono *una coppia, perché spezzarla???*

© UFFICIO COMPLICAZIONE COSE SEMPLICI

La solita vecchia storia: **rappresentazione naif = più codice**
Approccio migliore: **rappresentazione più sofisticata = meno codice**



IL SISTEMA SOFTWARE

Verso il refactoring

- Come nell'esempio banconote *manca il portafoglio..*
- ..qui manca *come minimo la coppia (partito, voti ottenuti)*
 - NON SOLO: nell'algoritmo si devono accoppiare anche *partito* e *resti*
- Per non parlare del fatto che *il risultato* è espresso in termini di *terne (partito, voti ottenuti, seggi assegnati)*

AARGH!! è successo di nuovo!

Ci siamo persi l'elefante!!

Mancano tutte le macro-strutture, abbiamo solo i pezzi!

Ci siamo fatti fregare di nuovo! Non dovrà più succedere!
Nuovo obiettivo: *più rappresentazione = meno codice*

REFACTORING

- Servono entità per accoppiare *partito*, *voti* [e *seggi*]

- 1^a possibilità:

- classe **PartitoConVoti** per la coppia (partito, voti)
 - classe **PartitoConSeggi** per la coppia (partito, seggi)
 - classe **PartitoConResti** per la coppia (partito, resti)

Modello pulito,
ma pletorico

- 2^a possibilità:

- classe **RisultatoDelPartito** per la **terna** (partito, voti, seggi)
 - classe **RestoDelPartito** per la **coppia** (partito, resti)

Buon
compromesso

- 3^a possibilità:

- classe **RisultatoDelPartito** per la **quadrupla** (partito, voti, seggi, **resti**)

Mischia dati
interni ed esterni

- Punti di attenzione:

- **partito e voti sono noti a priori, seggi e resti invece si calcolano dopo**
 - i seggi sono il risultato del calcolo → da fornire all'utente
 - i resti invece sono solo un "sottoprodotto" dell'algoritmo → non dovrebbero uscire

IL SISTEMA SOFTWARE

Seconda versione

- Architettura del sistema
 - nella sostanza invariata, *ma il metodo **calcolaSeggi** cambia signature*
 - non riceve più cose spezzettate, ma *un array di RisultatoDelPartito*
 - non restituisce più un array ad hoc, *agisce su quell'unico array*
 - al suo interno sfrutta un ulteriore array di RestoDelPartito

model::CalcolatoreSeggi
- nome: String
- sb: StringBuilder
+ calcolaSeggi(long, RisultatoDelPartito[]): void
+ CalcolatoreSeggi(String, StringBuilder)
+ getLogger(): StringBuilder
+ getNome(): String
- getVotiTotali(RisultatoDelPartito[]): long
- incSeggiPartito(String, RisultatoDelPartito[]): void
- trovaMax(RestoDelPartito[]): RestoDelPartito

model::RestoDelPartito
- partito: String
- resto: long
+ azzerareResto(): void
+ getPartito(): String
+ getResto(): long
+ RestoDelPartito(String, long)
+ toString(): String

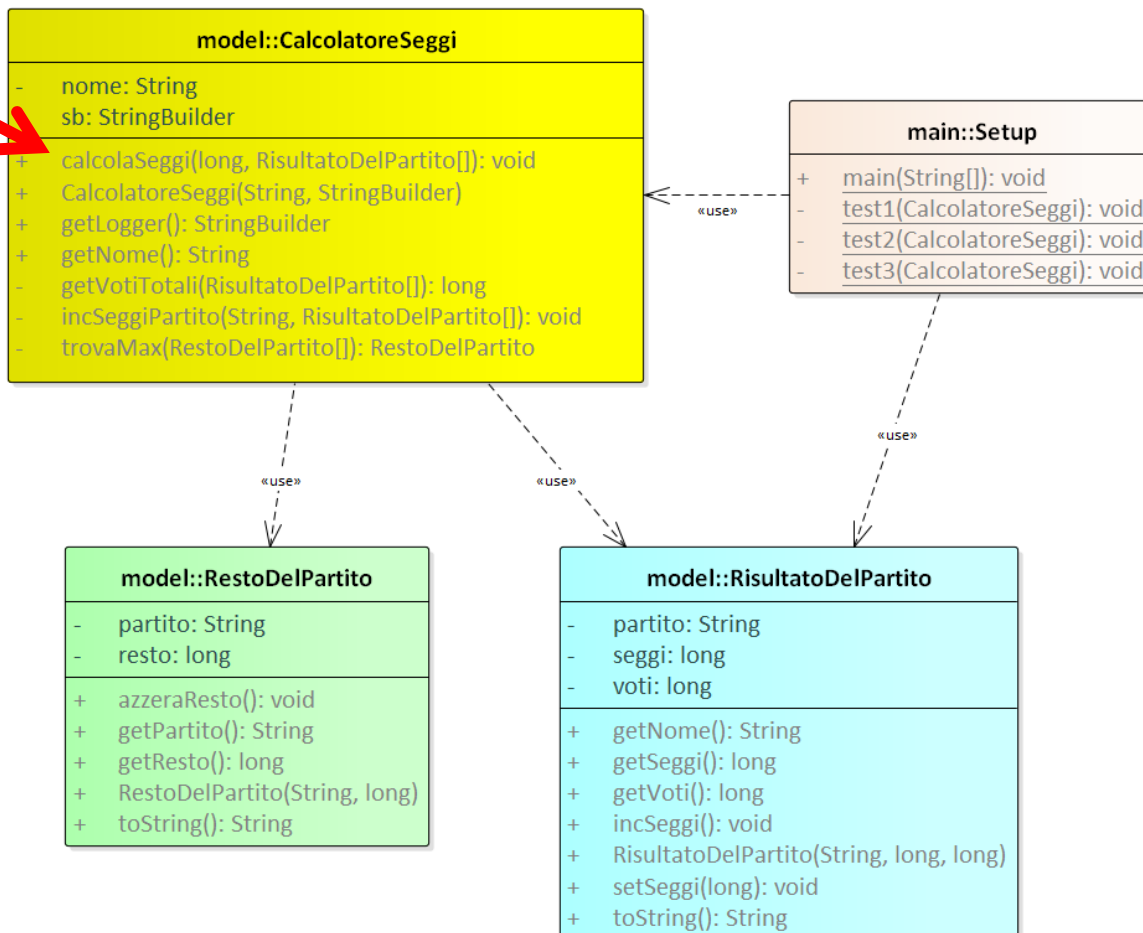
model::RisultatoDelPartito
- partito: String
- seggi: long
- voti: long
+ getNome(): String
+ getSeggi(): long
+ getVoti(): long
+ incSeggi(): void
+ RisultatoDelPartito(String, long, long)
+ setSeggi(long): void
+ toString(): String

calcolaSeggi(long, RisultatoDelPartito[]): void

IL SISTEMA SOFTWARE

Seconda versione

calcolaSeggi(long, RisultatoDelPartito[]): void





RAPPRESENTAZIONE DEI DATI

Java

```
package elections.model;

public class RisultatoDelPartito {
    private String partito;
    private long voti, seggi;

    public RisultatoDelPartito(String nome, long voti, long seggi) {
        this.partito = nome; this.voti = voti; this.seggi = seggi;
    }

    public long getSeggi() { return seggi; }
    public void incSeggi() { this.seggi++; }
    public void setSeggi(long seggi) { this.seggi=seggi; }
    public String getNome() { return partito; }
    public long getVoti() { return voti;}
    public String toString() {
        return partito + ",\tvoti=" + voti + ",\tseggi=" + seggi;
    }
}
```



RAPPRESENTAZIONE DEI DATI

Java

```
package elections.model;

public class RestoDelPartito {

    private String partito;
    private long resto;

    public RestoDelPartito(String partito, long resto) {
        this.partito = partito; this.resto = resto;
    }

    public String getPartito() { return partito; }
    public long getResto() { return resto; }
    public void azzeraresto() { resto=0; }

    public String toString() {
        return "RestoDelPartito [partito=" + partito + ", resto=" + resto + "]\n";
    }
}
```

ALGORITMO (1/2)

Java

```
package elections.model;

public void calcolaSeggi(long seggiDaAssegnare, RisultatoDelPartito[] partiti) {
    long votiTotali = getVotiTotali(partiti);
    double quoziente = (double) votiTotali / seggiDaAssegnare;
    sb.setLength(0); // clear the buffer
    for (RisultatoDelPartito p : partiti) {
        sb.append(p + System.lineSeparator());
    }
    sb.append("Seggi da assegnare: " + seggiDaAssegnare + System.lineSeparator());
    sb.append("Quoziente elettorale: " + quoziente + System.lineSeparator());

    int seggiAssegnati = 0;
    int indexResti = 0;
    RestoDelPartito[] resti = new RestoDelPartito[partiti.length];
    for (RisultatoDelPartito p : partiti) {
        p.setSeggi( (long) Math.floor(p.getVoti() / quoziente));
        seggiAssegnati += p.getSeggi();
        resti[indexResti++] = new RestoDelPartito(
            p.getNome(), (long) (p.getVoti() % quoziente));
    }
    ...
}
```

Stampe senza indici
grazie al fatto che
ogni partito sa i suoi
voti (e i suoi seggi)

Calcoli senza indici
per lo stesso motivo

tranne indexResti
che dà la dimensione
logica (array usato
come lista)

(eliminabile un domani usando una vera lista)

ALGORITMO (2/2)

Java

```
...
sb.append("Seggi assegnati al primo giro: " + System.lineSeparator());
for (RisultatoDelPartito p : partiti) {
    sb.append(p + System.lineSeparator());
}
sb.append("Resti dopo il primo giro: " + System.lineSeparator());
for (RestoDelPartito r : resti) {
    sb.append(r + System.lineSeparator());
}
sb.append("Seggi ancora da assegnare: " +
    (seggiDaAssegnare-seggiAssegnati) + System.lineSeparator());

while (seggiDaAssegnare > seggiAssegnati) {
    RestoDelPartito max = trovaMax(resti);
    incSeggiPartito(max.getPartito(), partiti);
    seggiAssegnati++;
    max.azzeraresto();
}

sb.append("Seggi assegnati alla fine: " + System.lineSeparator());
for (RisultatoDelPartito p : partiti) {
    sb.append(p + System.lineSeparator());
}
}
```

Altre stampe..

Metodi ausiliari

Metodi ausiliari

Calcoli senza indici
grazie al fatto che
ogni partito sa i suoi
voti (e i suoi seggi)

Altre stampe..

COLLAUDO ARTIGIANALE BIS

```
private static void test1(CalcolatoreSeggi calculator) {  
    RisultatoDelPartito[] partiti = {  
        new RisultatoDelPartito("Formiche rosse", 10, -1),  
        new RisultatoDelPartito("Topolini grigi", 100, -1),  
        new RisultatoDelPartito("Farfalle blu", 200, -1),  
        new RisultatoDelPartito("Bruchi verdi", 300, -1),  
        new RisultatoDelPartito("Canarini ocra", 400, -1)  
    };  
    long[] seggiAttesi = {1,10,20,30,39};;  
    assert...;  
    ...  
}
```

Java

Problema: cosa
mettere all'inizio
come seggi?

**Ci vorrebbe NaN ma
Long e Integer
non ce l'hanno!**

Occorre un valore che *non si possa
confondere con un valore vero*
Visto che i seggi non possono essere
meno di 0, scegliamo di usare **-1**

```
Formiche rosse,  voti=10,  seggi=1  
Topolini grigi,  voti=100, seggi=10  
Farfalle blu,   voti=200, seggi=20  
Bruchi verdi,   voti=300, seggi=30  
Canarini ocra,  voti=400, seggi=39
```

```
---Log calcolo---  
Formiche rosse,  voti=10,  seggi=-1  
Topolini grigi,  voti=100, seggi=-1  
Farfalle blu,   voti=200, seggi=-1  
Bruchi verdi,   voti=300, seggi=-1  
Canarini ocra,  voti=400, seggi=-1  
Seggi da assegnare: 100  
...
```



COLLAUDO ARTIGIANALE BIS

Il log completo è più lungo perché le stampe coi cicli sugli array usano le `toString` delle varie classi, che sono piuttosto dettagliate

```
---Log calcolo---
Formiche rosse,  voti=10,  seggi=-1
Topolini grigi,  voti=100, seggi=-1
Farfalle blu,    voti=200, seggi=-1
Bruchi verdi,    voti=300, seggi=-1
Canarini ocra,   voti=400, seggi=-1
Seggi da assegnare: 100
Quoziente elettorale : 10.1
Seggi assegnati al primo giro:
Formiche rosse,  voti=10,  seggi=0
Topolini grigi,  voti=100, seggi=9
Farfalle blu,    voti=200, seggi=19
Bruchi verdi,    voti=300, seggi=29
Canarini ocra,   voti=400, seggi=39
Resti dopo il primo giro:
RestoDelPartito [partito=Formiche rosse, resto=10]
RestoDelPartito [partito=Topolini grigi, resto=9]
RestoDelPartito [partito=Farfalle blu, resto=8]
RestoDelPartito [partito=Bruchi verdi, resto=7]
RestoDelPartito [partito=Canarini ocra, resto=6]
Seggi ancora da assegnare: 4
Seggi assegnati alla fine:
Formiche rosse,  voti=10,  seggi=1
Topolini grigi,  voti=100, seggi=10
Farfalle blu,    voti=200, seggi=20
Bruchi verdi,    voti=300, seggi=30
Canarini ocra,   voti=400, seggi=39
```

IL SISTEMA SOFTWARE

Va bene così?

- Per quello che possiamo fare ora, sì
- Però, la difficoltà di rappresentare i dati ha messo in luce che **coi soli array si fatica a catturare relazioni non banali**
 - MOTIVO: l'array cattura bene solo il concetto di *sequenza*
- In questo caso, sarebbe stato molto comodo poter disporre di una **struttura dati a tabella** con cui esprimere *nativamente* l'idea di *relazione fra N entità*
 - come infatti (ma guarda..!) l'analisi *suggeriva fin dall'inizio*:

	Voti ottenuti	Seggi al quoziente	Seggi assegnati	Resti	Seggi ai resti	Tot. Seggi
Lista A	171.000	19,00	19	0	0	19
Lista B	132.000	14,67	14	6.000	1	15
Lista C	84.000	9,33	9	3.000	0	9
Lista D	36.000	4,00	4	0	0	4
	423.000		46			47

IL SISTEMA SOFTWARE

Va bene così?

- Possiamo averla?
 - ovviamente sì!
 - MA costruircela a mano è faticoso...
 - ..e probabilmente inutile, *visto che non saremo certo gli unici né i primi ad avere questa necessità!*
- Scopriremo fra qualche tempo che **Java Collection Framework** offre a tale scopo l'astrazione **Mappa**
 - OBIETTIVO: catturare bene il concetto di *tabella a due colonne*

`ogg = get(key2)`
restituisce *oggetto2*

<i>chiave</i>	<i>valore</i>
key1	oggetto1
key2	oggetto2
...	...

Se le colonne sono più di due, basta usare come valore un *oggetto composto*

`put(key3, oggetto3)`

E SE VOLESSIMO UN DIVERSO SISTEMA ELETTORALE?

- Purché il sistema rimanga **proporzionale**, cambiare algoritmo è facile – basta scrivere un nuovo `calcolatoreSeggi`!
 - sistema D'Hondt
 - sistema Saint-Lague
- Ad esempio, il sistema D'Hondt:
 - divide i voti di ogni partito per 1,2,3,4,5,6,7...
 - assegna i seggi ai vari partiti partendo dai *risultati più alti*

Partiti	Voti	1	2	3	4	5	Seggi
Lista A	171.000	171000	85500	57000	42750	34200	5
Lista B	132.000	132000	66000	44000	33000	26400	4
Lista C	84.000	84000	42000	28000	21000	8400	2
Lista D	36.000	36000	18000	12000	9000	3600	1

Qui i seggi da assegnare sono 12 → si prendono i 12 risultati più alti (in grassetto).
Ciò porta ad assegnare 5 seggi alla Lista A, 4 seggi alla Lista B, 2 alla Lista C, etc.

Come lo modellereste..?

E SE VOLESSIMO UN DIVERSO SISTEMA ELETTORALE?

- Il sistema D'Hondt è solo uno dei vari **sistemi a divisore**
 - nel sistema Saint-Lague, i divisori sono 1,3,5,7,9...
 - in altri sistemi si utilizzano serie di divisori ancora diverse

→ *meglio non cablare nel codice i divisori!*
- Se invece il sistema elettorale fosse **maggioritario**
 - ossia, tale che in ogni collegio *"the winner takes all"*
 - non basterebbe più avere solo il totale dei voti ottenuti da un partito
- occorrerebbe conoscere la **distribuzione dei voti nei collegi**
 - servirebbe, cioè, un dato *a granularità più fine* di quello, a grana grossa, sufficiente nel proporzionale
 - UN GIORNO potremo anche avere tutto ciò su file e leggerlo.. 😊
(ma quel giorno non è oggi!)